Chapter 14

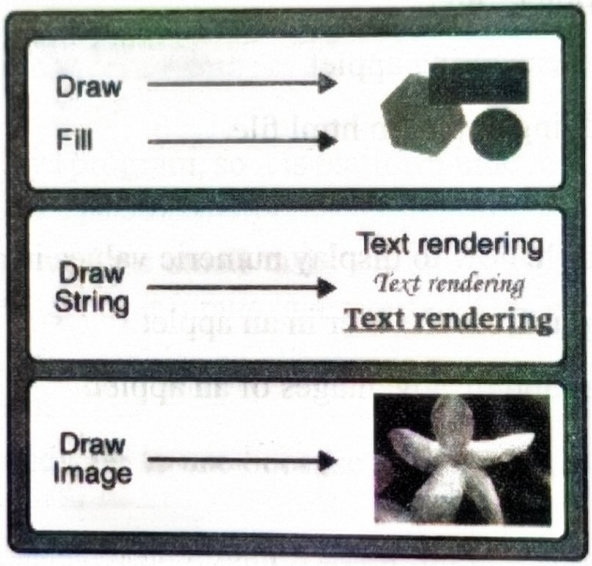
GRAPHICS PROGRAMMING

14.1 Introduction

Most methods of the Graphics class can be divided into two basic groups:

- **Draw and fill methods:** enabling you to render basic shapes, text and images.
- **Attributes setting methods:** which affect how that drawing and filling appears.

The following figure shows how these methods relate to graphic objects:



Every applet has its own area of the screen known as canvas, where it creates its display. The size of an applet's space is decided by the attributes of the <APPLET> tag. A java applet draws graphical image inside its space using the coordinate system as shown in figure 14.1

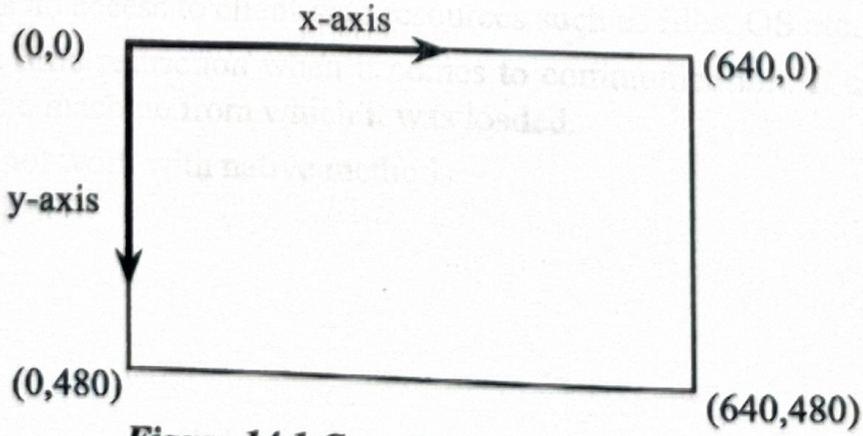


Figure 14.1 Coordinate system of Java

14.2 The Graphics Class

The *Graphics* class defines a number of drawing functions, each shape can be drawn edge-only or filled. To draw shapes on the screen, we may call one of the methods available in the *Graphics* class. To draw a shape, we only need to use the appropriate method with the required arguments.

The most commonly used drawing methods included in the *Graphics* class are listed below:

Method	Description
clearRect()	Erase a rectangular area of the canvas.
copyAre()	Copies a rectangular area of the canvas to another area
drawArc()	Draws a Arc
drawLine()	Draws a straight line
drawOval()	Draws a Oval
drawPolygon()	Draws a polygon
drawRect()	Draws a rectangle
drawRoundRect()	Draws a rectangle with rounded corners
drawString()	Display a text string
fillArc()	Draws a filled arc
fillOval()	Draws a filled Oval
fillPolygon()	Draws a filled Polygon
fillRect()	Draws a filled rectangle
fillRoundRect()	Draws a filled rectangle with rounded corners
getColor()	Retrieves the current drawing color
getFont()	Retrieves the currently used font
getFontMetrics()	Retrieves the information about the current font
setColor()	Sets the drawing color
setFont()	Sets the font

14.3 Lines and Rectangles

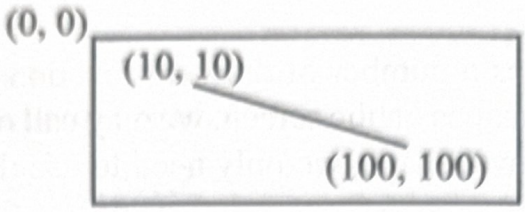
Lines

A line is a simple graphics primitive. A line is an object which connects two points. Lines are drawn with the **`drawLine()`** method.

Syntax:

```
drawLine(int x1, int y1, int x2, int y2);
```

The following program shows how to draw straight *line* from the coordinate point (10,10) to (100,100) in an applet will be saved with named *ExLine.java:*

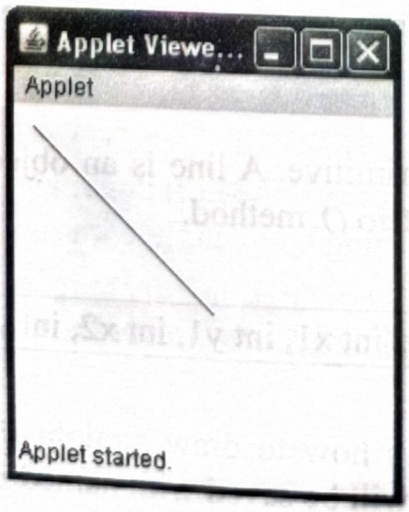



```
//Drawing Line
import java.awt.*;
import java.applet.*;
public class ExLine extends Applet
{
 public void paint(Graphics g)
 {
 g.drawLine(10,10,100,100);
 }
}
```

The compiled output file called *ExLine.class* should be placed in the same directory as the source file. After creating *ExLine.class* file, we need create *ExLine.html* file and run that as shown below:

```
<Html>
<Head>
<Title> Line between two coordinates </Title>
</Head>
<Body>
<Center>
<Applet code=ExLine.class  Width=400  Height=400>
</Applet>
</Center>
</Body>
</Html>
```

Output



Rectangles

In Java, to draw a *rectangle* or *filled rectangle*, to draw a *rounded rectangle* or filled *rounded rectangle*, we can use the following methods provided by the ***Graphics*** class:

Methods	Description
drawRect()	Draws a rectangle
fillRect()	Draws a filled rectangle
drawRoundRect()	Draws a rectangle with rounded corners
fillRoundRect()	Draws a filled rectangle with rounded corners

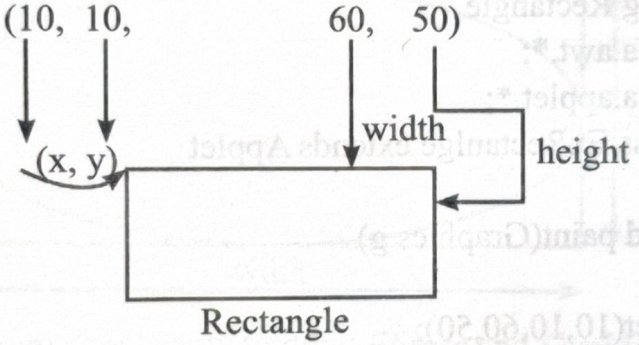
drawRect() & fillRect()

Syntax:

```
drawRect(int x, int y, int width, int height);
fillRect(int x, int y, int width, int height);
```

Here,

- (x, y) is the upper left corner of the rectangle.
- width and height: specify dimension of the rectangle.



drawRoundRect() & fillRoundRect()

Syntax:

```
drawRoundRect(int x, int y, int width, int height, int arcWidth, int arcHeight);
fillRoundRect(int x, int y, int width, int height, int arcWidth, int arcHeight);
```

Here,

- (x, y) is the upper left corner of the rectangle.
- width and height: specify dimension of the rectangle.
- arcWidth and arcHeight: specify the horizontal and vertical diameters of the arcs at the four corners when drawing a rectangle with rounded corners.

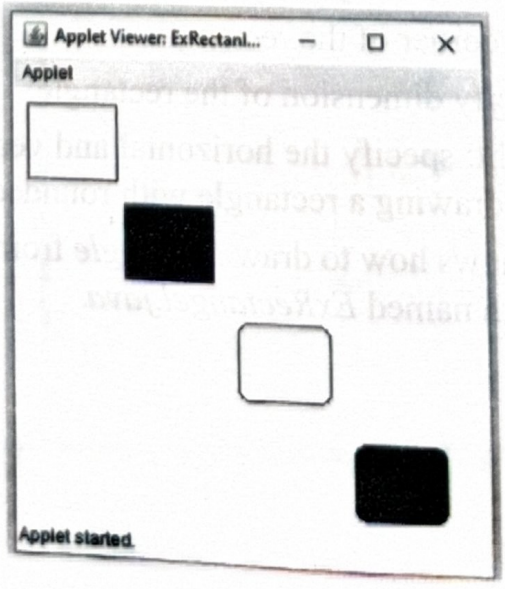
The following program shows how to draw *Rectangle* from the given coordinate values in an applet will be saved with named *ExRectangel.java*.

```
// Drawing Rectangle
import java.awt.*;
import java.applet.*;
public class ExRectanlge extends Applet
{
 public void paint(Graphics g)
 {
 g.drawRect(10,10,60,50);
 g.fillRect(100,100,100,0);
 g.drawRoundRect(190,10,60,50,15,15);
 g.fillRoundRect(70,90,140,100,30,40);
 }
}
```

The compiled output file called *ExRectangle.class* should be placed in the same directory as the source file. After creating *ExRectangle.class* file, we need create *ExRectangle.html* file and run that as shown below:

```
//  Drawing Rectangle
import java.awt.*;
import java.applet.*;
public class ExRectanlge extends Applet
{
public void paint(Graphics g)
{
g.drawRect(10,10,60,50);
g.fillRect(75,75,60,50);
g.drawRoundRect(150,150,60,50,15,15);
g.fillRoundRect(225,225,60,50,15,15);
}
}
```

Output



14.4 Circles and Ellipses

In Java, to draw a *circle or filled circle, ellipse or filled ellipse*, we can use the following methods provided by the ***Graphics*** class. The Graphics class does not contain method for *circles or ellipses.*

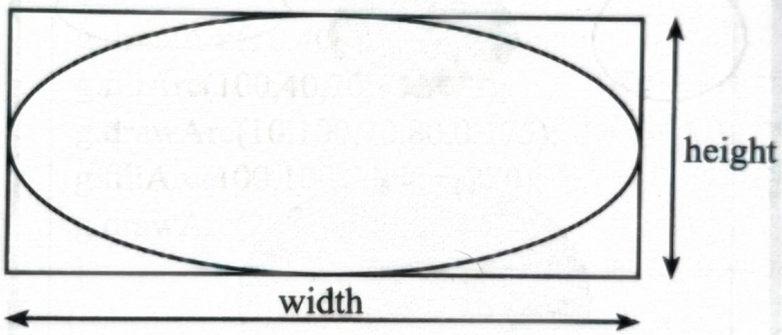
Methods	Description
drawOval()	Draws a Oval
fillOval()	Draws a filled Oval

Syntax:

```
drawOval(int x, int y, int width, int height);
fillOval(int x, int y, int width, int height);
```

Here,

- (x, y) is the upper left corner of the circle or oval.
- width and height: specify dimension of the circle or oval.



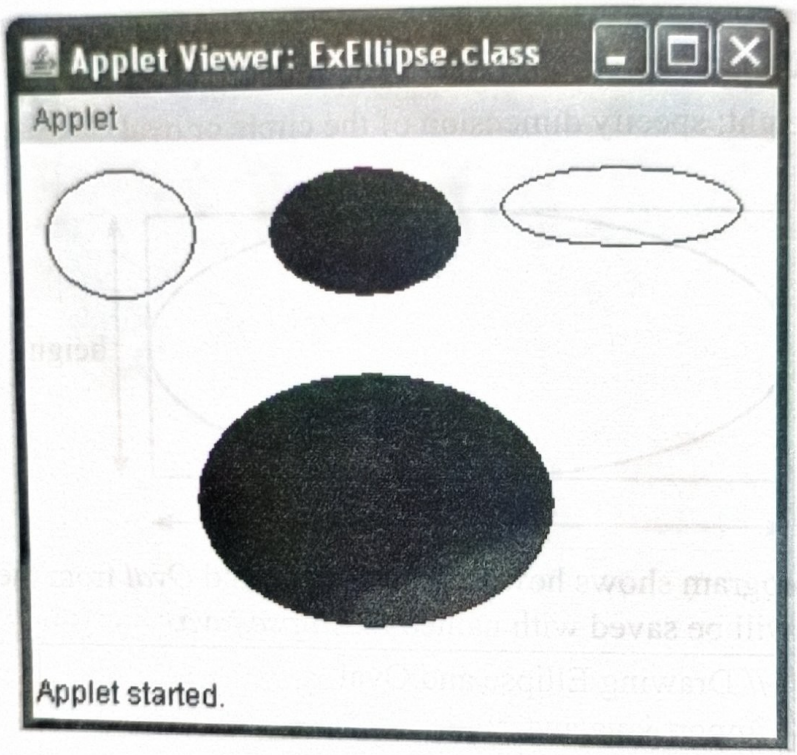
The following program shows how to draw *Ellipse* and *Oval* from the given coordinate values in an applet will be saved with named *ExEllipse.java.*

```
// Drawing Ellipse and Oval
import java.awt.*;
import java.applet.*;
public class ExEllipse extends Applet
{
 public void paint(Graphics g)
 {
 g.drawOval(10,10,60,50);
 g.fillOval(100,10,75,50);
 g.drawOval(190,10,90,30);
 g.fillOval(70,90,140,100);
 }
}
```

The compiled output file called *ExEllipse.class* should be placed in the same directory as the source file. After creating *ExEllipse.class* file, we need create *ExEllipse.html* file and run that as shown below:

```
<Html>
<Head>
   <Title> Drawing Ellipse and Oval </Title>
</Head>
<Body>
<Center>
<Applet code=ExEllipse.class  Width=400  Height=400>
</Applet>
</Center>
</Body>
</Html>
```

Output



14.5 Drawing Arcs

An arc is a part of oval. In Java, to draw a ***Arc or filled Arc***, we can use the following methods provided by the ***Graphics*** class.

Methods	Descriptions
drawArc()	Draws a Arc
fillArc()	Draws a filled arc

Syntax:

```
drawArc(int x, int y, int width, int height, int startAngle, int sweepAngle);
fillArc(int x, int y, int width, int height, int startAngle, int sweepAngle);
```

Here,

- (x, y) is the upper left corner of the Arc.
- width and height: specify dimension of the Arc.
- starAngle and sweepAngle: The arc is drawn from startAngle through the angular distance specified by sweepAngle. Angles are specified in degree. '0°' is on the horzontal, at the 30' clock's position. The arc is drawn conter clockwise if sweepAngle is positive, and clockwise if sweepAngle is negative.

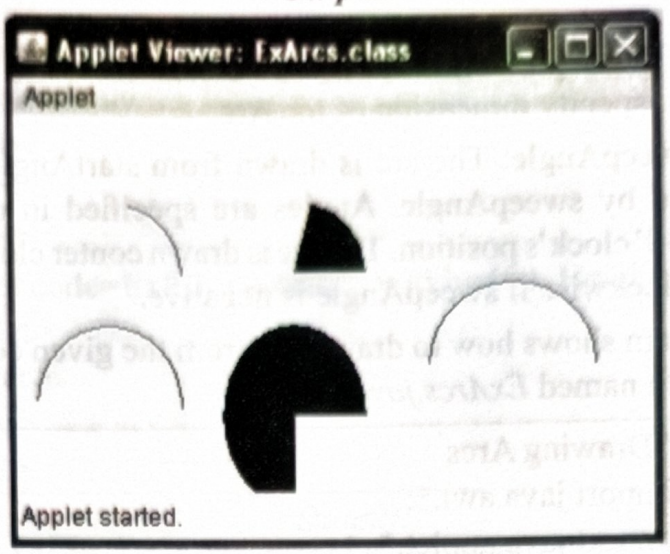
The following program shows how to draw *Arcs* from the given coordinate values in an applet will be saved with named *ExArcs.java.*

```
//Drawing Arcs
import java.awt.*;
import java.applet.*;
public class ExArcs extends Applet
{
 public void paint(Graphics g)
 {
 g.drawArc(10,40,70,70,0,75);
 g.fillArc(100,40,70,70,0,75);
 g.drawArc(10,100,70,80,0,175);
 g.fillArc(100,100,70,90,0,270);
 g.drawArc(200,80,80,80,0,180);
 }
}
```

The compiled output file called *ExArcs.class* should be placed in the same directory as the source file. After creating *ExArcs.class* file, we need create *ExArcs.html* file and run that as shown below:

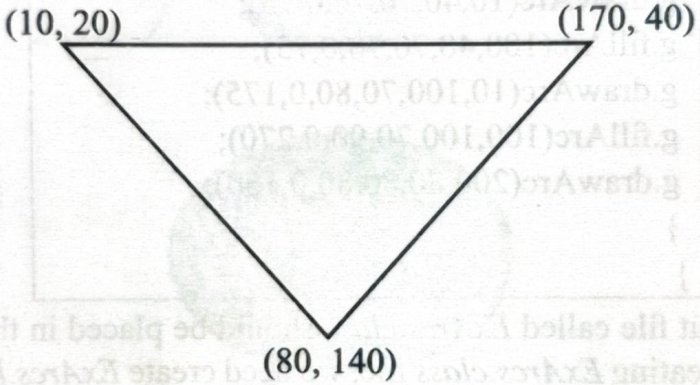
```
<Html>
<Head>
<Title> Drawing Arcs </Title>
</Head>
<Body>
<Center>
<Applet code=ExArcs.class  Width=400  Height=400>
</Applet>
</Center>
</Body>
</Html>
```

Output



14.6 Drawing Polygons

Polygons are shapes with many sides. It may be considered a set of lines connected together. The end of the first line is the beginning of the second line, the end of the second line is the beginning of the third line, and so on.



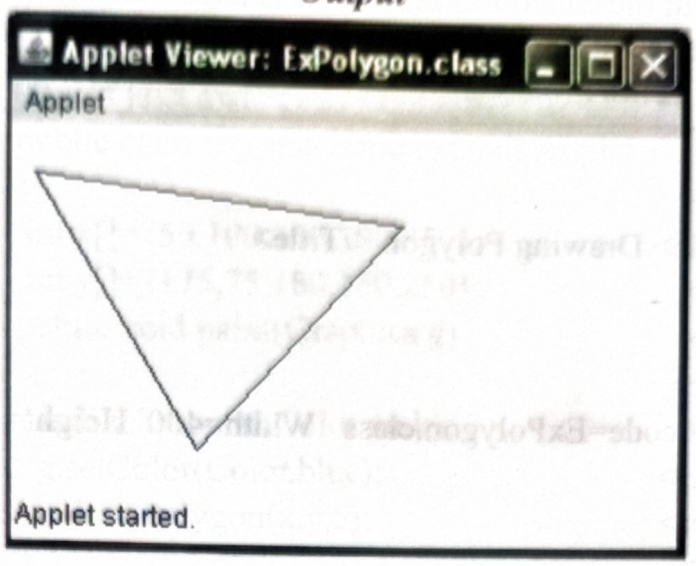
A polygon with three sides

The following program shows how to draw polygon from the given coordinates:

```
//Drawing Polygon
import java.awt.*;
import java.applet.*;
public class ExPolygon extends Applet
{
 public void paint (Graphics g)
{
  g.drawLine(10,20,170,40);
  g.drawLine(170,40,80,140);
  g.drawLine(80,140, 10,20);
}
}
```

```
<Html>
<Head>
<Title> Drawing Polygon </Title>
</Head>
<Body>
<Center>
<Applet code=ExPolygon.class
Width=400 Height=400>
</Applet>
</Center>
</Body>
</Html>
```

Output



In Java, to draw a polygon or to draw a filled polygon, we can use the following methods provided by the Graphics class:

Methods	Descriptions
drawPolygon()	Draws a polygon
fillPolygon()	Draws a filled Polygon

Syntax:

```
drawPolygon(int x[], int y[], int numpoints);
fillPolygon(int x[], int y[], int numpoints);
```

Here,

- (x[], y[]), The polygon's endpoints are specified by the coordinate pairs contained within the x and y arrays. It is obvious that x and y arrays should be of the same size and we must repeat the first point at the end of the array for closing the polygon.
- numpoints: The number of points defined by x and y is specified by numPoints.

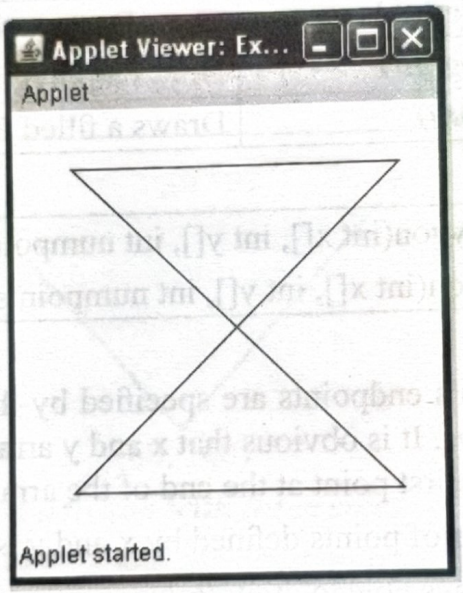
The following programshows how to draw *Polygon* from the given coordinate values in an applet will be saved with named *ExPolygon.java.*

```
//Drawing Polygon
import java.awt.*;
import java.applet.*;
public class ExPolygon extends Applet
{
public void paint(Graphics g)
{
int xpoints[]={30,200,30,200,30};
int ypoints[]={30,30,200,200,30};
int num=5;
g.drawPolygon(xpoints,ypoints,num);
} }
```

The compiled output file called *ExPolygon.class* should be placed in the same directory as the source file. After creating *ExPolygon.class* file, we need create *ExPolygon.html* file and run that as shown below:

```
<Html>
<Head>
   <Title> Drawing Polygon </Title>
</Head>
<Body>
<Center>
<Applet code=ExPolygon.class  Width=400  Height=400>
</Applet>
</Center>
</Body>
</Html>
```

Output



14.7 Line Graphs

A *line chart* or *line graph* displays information as a series of data points connected by straight line segments. Line Chart shows how the data changes at equal time frequency.

- A line graph represents the relationship between two variables graphically.
- We can implement this relationship using the ***drawPolygon()*** method of the Graphics class.

The following programshows how to draw *LineGraph* from the given table of values in an applet will be saved with named *ExLineGraph.java.*

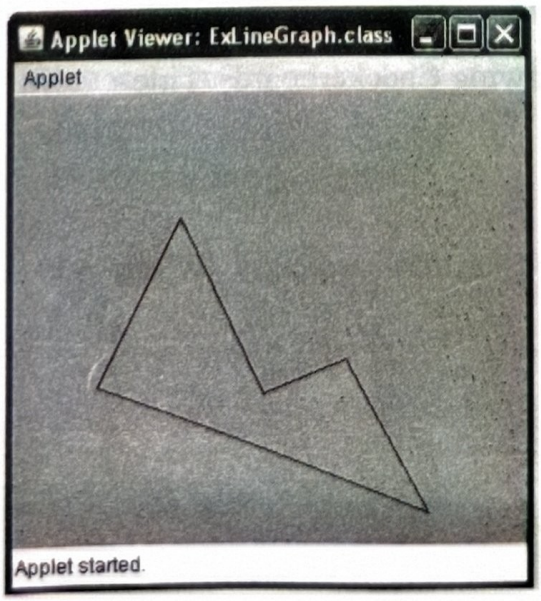
X	50	100	150	200	250
Y	175	75	180	160	250

```
// Line Graph
import java.applet.*;
import java.awt.*;
public class ExLineGraph extends Applet
{
 int x[]={50,100,150,200,250};
 int y[]={175,75,180,160,250};
public void paint(Graphics g)
 {
  setBackground(Color.pink);
  g.setColor(Color.blue);
  g.drawPolygon(x,y,5);
}
}
```

The compiled output file called *ExLineGraph.class* should be placed in the same directory as the source file. After creating *ExLineGraph.class* file, we need create *ExLineGraph.html* file and run that as shown below:

```
<Html>
<Head>
    <Title> Drawing Line Graph </Title>
</Head>
<Body>
<Center>
<Applet code=ExLineGraph.class   Width=400  Height=400>
</Applet>
</Center>
</Body>
</Html>
```

Output



14.8 Using Control Loops in Applets

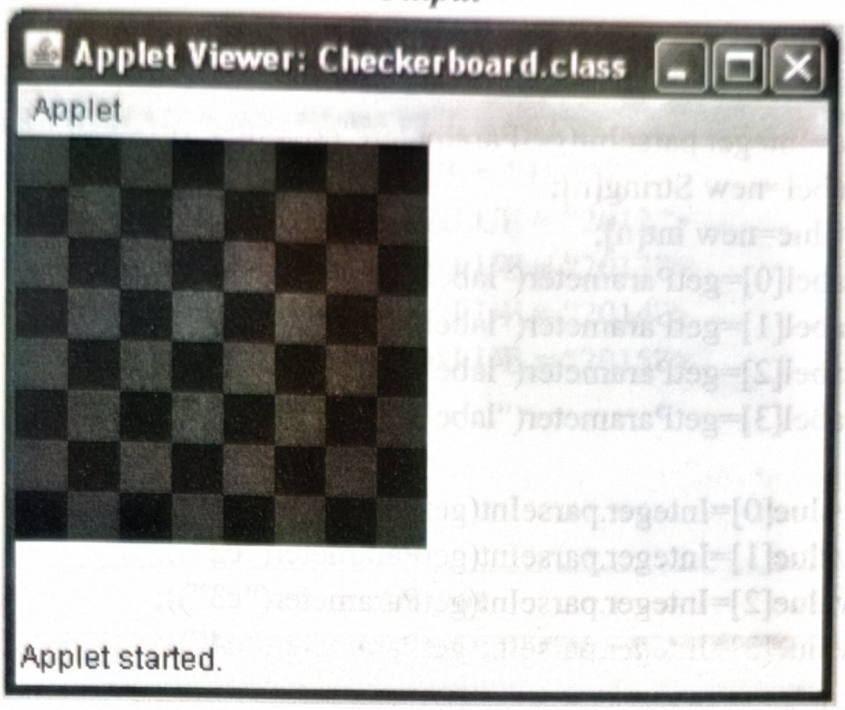
We can use all control structures in an applet. The following program uses *a* ***for*** loop for drawing squares repeatedly in an applet will be saved with named *Checkerboard.java.*

```
// Control Loops
import java.awt.*;
import java.applet.*;
public class Checkerboard extends Applet
 {
 public void paint(Graphics g)
 {
 int row; // Row number, from 0 to 7
 int col; // Column number, from 0 to 7
 int x,y; // Top-left corner of square
 for ( row = 0; row < 8; row++ )
 {
 for ( col = 0; col < 8; col++)
 {
 x = col * 20;
 y = row * 20;
 if ( (row % 2) == (col % 2) )
 g.setColor(Color.red);
 else
 g.setColor(Color.black);
 g.fillRect(x, y, 20, 20);
 } } } }
```

The compiled output file called *Checkerboard.class* should be placed in the same directory as the source file. After creating *Checkerboard.class* file, we need create *Checkerboard.html* file and run that as shown below:

```
<Html>
<Head>
   <Title> Drawing Checkerboard</Title>
</Head>
<Body>
<Center>
<Applet code=Checkerboard.class  Width=400  Height=400>
</Applet>
</Center>
</Body>
</Html>
```

Output



14.9 Drawing Bar Charts

Applets can be designed to display bar charts, which are commonly used in comparative analysis of data. The table below shows the result analysis of computer science department during the period 2012 to 2015. These values may be placed in a HTML file as PARAM attributes and then used in an applet for displaying a bar chart.

Year	2012	2013	2014	2015
Result Analysis (Maximum 100%)	90	95	93	100

The method **getParameter()** is used to fetch the data values from the HTML file. The method **getParameter()** returns only string values and therefore we use wrapper class method parseInt() to convert strings to integer values.

The following program shows how to draw *Bar Charts* from the given coordinate values in an applet will be saved with named *ExBarchart.java.*

```
// Program for drawing Bar Chart
import java.awt.*;
import java.applet.*;
public class ExBarchart extends Applet
{
int n=0;
String label[];
int value[];
public void init()
```

```
{
try
{
 n=Integer.parseInt(getParameter("columns"));
label=new String[n];
value=new int[n];
label[0]=getParameter("label1");
label[1]=getParameter("label2");
label[2]=getParameter("label3");
label[3]=getParameter("label4");

value[0]=Integer.parseInt(getParameter("c1"));
value[1]=Integer.parseInt(getParameter("c2"));
value[2]=Integer.parseInt(getParameter("c3"));
value[3]=Integer.parseInt(getParameter("c4"));
  }
catch(NumberFormatException e)
{
  }
}
public void paint(Graphics g)
{
for(int i=0;i<n;i++)
{
g.setColor(Color.blue);
g.drawString(label[i],20,i*50+30);
g.fillRect(50,i*50+10,value[i],40);
}
}
}
```

The compiled output file called *ExBarchat.class* should be placed in the same directory as the source file. After creating *ExBarcahrt.class* file, we need create *ExBarchart.html* file and run that as shown below:

```
<Html>
<Head>
   <Title> Drawing Bar Chart </Title>
</Head>
<Body>
<Center>
<applet code="ExBarchart.class"  width=500     height=500
```

```
<PARAM NAME = "columns" VALUE = "4">
<PARAM NAME = "c1" VALUE = "90">
<PARAM NAME = "c2" VALUE = "95">
<PARAM NAME = "c3" VALUE = "93">
<PARAM NAME = "c4" VALUE = "100">
<PARAM NAME = "label1" VALUE = "2012">
<PARAM NAME = "label2" VALUE = "2013">
<PARAM NAME = "label3" VALUE = "2014">
<PARAM NAME = "label4" VALUE = "2015">
</applet>
</Center>
</Body>
</Html>
```

Output



14.10 The repaint() Method

- We can call repaint() method when you want the applet area to be re drawn.
- The repaint() method calls the update().
- The default action of update() method is to clear the applet area and call the paint().

repaint versions:-

- *void repaint()*:- causes entire window to be repainted .
- *void repaint(int left, int top, int width, int height)* : Specifies region that will be repainted. Left and top are co-ordinates of upper left corner, width and height of the region.
- *void repaint(long maxDelay)*:-specifies maximum number of milliseconds that can elapse before update is called.

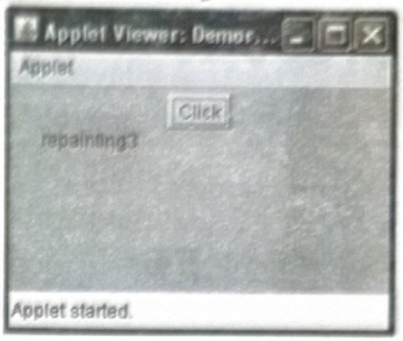
Example

The following program shows the usage of repaint() method:

```
// repaint() method
import java.awt.*;
import java.applet.*;
import java.awt.event.*;
public class Demorepaint extends Applet implements ActionListener
{
Button bt;
int i=0;
public void init()
{
bt=new Button("Click");
add(bt);
bt.addActionListener(this);
setBackground(Color.pink);
setForeground(Color.blue);
}
public void paint(Graphics g)
{
g.drawString("repainting"+i,20,40);
i++;
}
public void actionPerformed(ActionEvent ae)
{
repaint();
}
}
```

```
<Html>
<Head>
<Title> Repaint </Title>
</Head>
<Body>
<Center>
<Applet code=Demorepaint.class Width=400 Height=400>
</Applet>
</Center>
</Body>
</Html>
```

Output

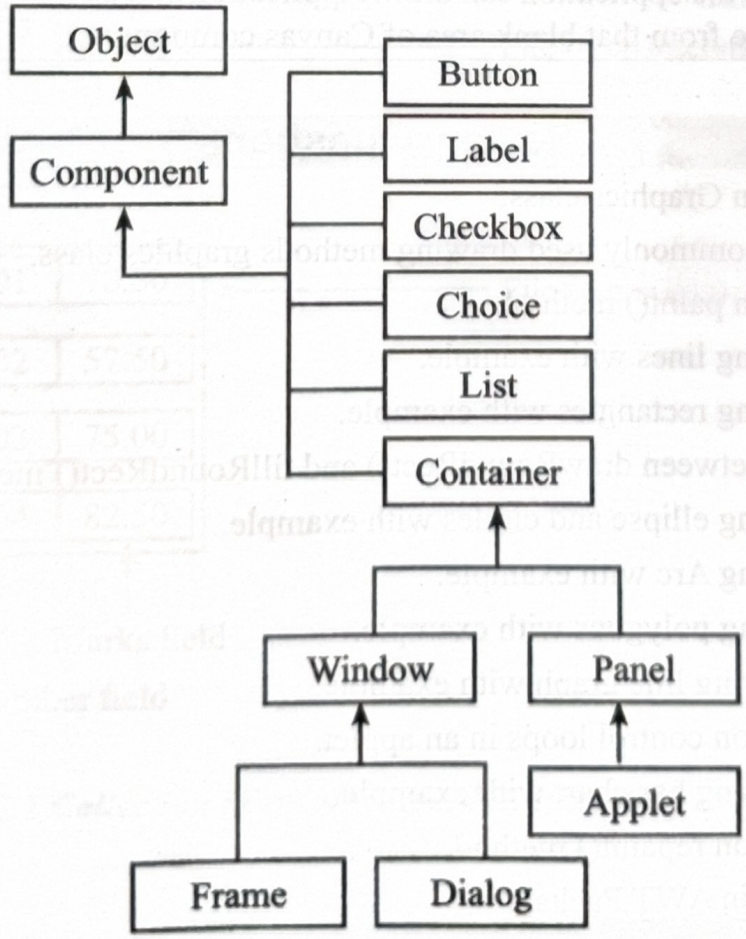


14.11 Introduction to AWT Packages

The Java AWT (Abstract Windowing Toolkit) is *an API to develop GUI or window-based application in java*. Java AWT components are platform-dependent i.e. components are displayed according to the view of operating system. AWT is heavyweight. i.e. its components uses the resources of system.

It contains a number of classes that help to implement common Windows based tasks, such as manipulating windows, adding scroll bars, buttons, list items, text boxes, etc. All the classes are contained in the ***java.awt*** package.

The hierarchy of Java AWT classes are given below:



Term	Description
Component	Component is an object having a graphical representation that can be displayed on the screen and that can interact with the user. For examples buttons, checkboxes, list and scrollbars of a graphical user interface.
Container	Container object is a component that can contain other components. Components added to a container are tracked in a list. The order of the list will define the components' front-to-back stacking order within the container. If no index is specified when adding a component to a container, it will be added to the end of the list.
Panel	Panel provides space in which an application can attach any other components, including other panels.
Window	Window is a rectangular area which is displayed on the screen. In different window we can execute different program and display different data. Window provide us with multitasking environment. A window must have either a frame, dialog, or another window defined as its owner when it's constructed.
Frame	A Frame is a top-level window with a title and a border. The size of the frame includes any area designated for the border. Frame encapsulates **window**. It and has a title bar, menu bar, borders, and resizing corners.
Canvas	Canvas component represents a blank rectangular area of the screen onto which the application can draw. Application can also trap input events from the use from that blank area of Canvas component.

Review Questions

1. Write a note on Graphics class.
2. What are the commonly used drawing methods graphics class.
3. Write a note on paint() method.
4. Explain drawing lines with example.
5. Explain drawing rectangles with example.
6. Differentiate between drawRoundRect() and fillRoundRect() methods.
7. Explain drawing ellipse and circles with example.
8. Explain drawing Arc with example.
9. Explain drawing polygons with example.
10. Explain drawing line graph with example.
11. Write a note on control loops in an applet.
12. Explain drawing bar chart with example.
13. Write a note on repaint() method.
14. Briefly explain AWT Packages.
15. Differentiate between paint() and repaint() methods.